ASSIGNMENT-1

Name: P. Greeshma

Regno: 192424418

Course Code: DSA0120

Github Link : https://github.com/parvathareddygreeshma-png/DSA0120-Objected-programming-with-c-

Submission date: 09-11-2026

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession. The first version of the gate software was written in a hurry. Some fares are off by a paisa, the peak check accepts 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance once appeared on a card, and the fare values are typed in several places. The number of journeys processed may grow: 1 station (20,000 journeys a day) → 1 line (500,000 a day) → whole network (3,000,000 a day) You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants, operators and control structures, implement the fare logic and validate it, so that every fare and balance is exact. Design Challenge: The metro is adding a fourth zone, a weekend discount and monthly passes. Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

Program:

```
#include <array>

#include <iomanip>

#include <iostream>

#include <optional>

#include <string>

enum class Zone {

Inner,

Middle,

Outer,

Airport,

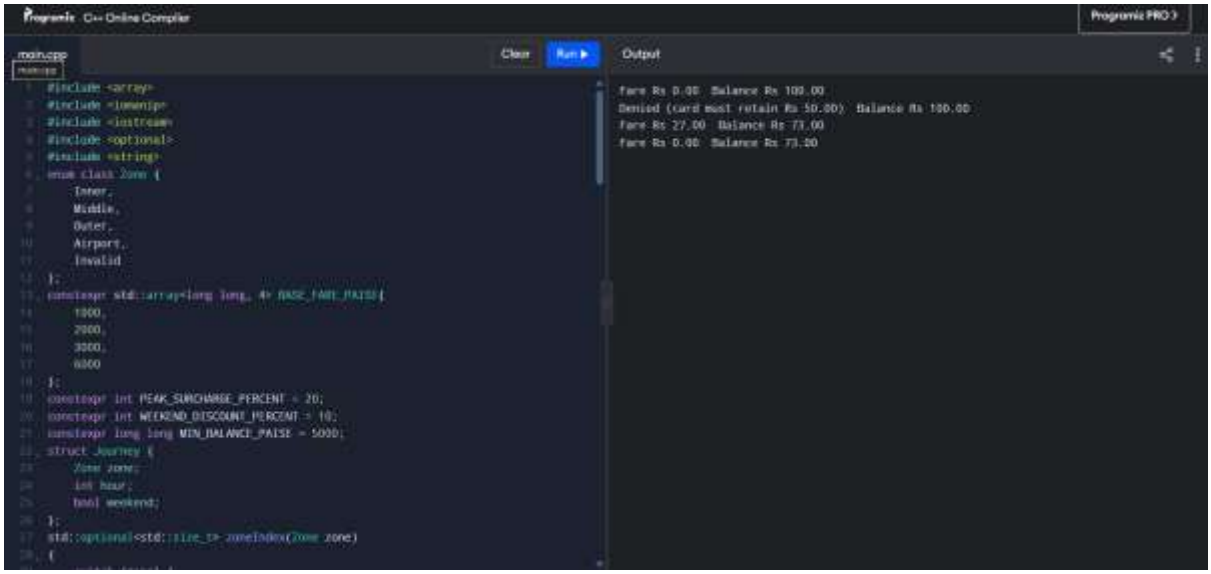
Invalid

};
```

```
constexpr std::array<long long, 4> BASE_FARE_PAISE{
1000,
2000,
3000,
6000
};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
constexpr long long MIN_BALANCE_PAISE = 5000;
struct Journey {
Zone zone;
int hour;
bool weekend;
};
std::optional<std::size_t> zoneIndex(Zone zone)
{
switch (zone) {
case Zone::Inner: return 0;
case Zone::Middle: return 1;
case Zone::Outer: return 2;
case Zone::Airport:return 3;
case Zone::Invalid:return std::nullopt;
}
return std::nullopt;
}
bool isPeakHour(int hour)
{
return (hour >= 8 && hour < 10) ||
(hour >= 17 && hour < 20);
```

```
}
class MonthlyPass {
private:
std::array<bool, 4> coveredZones{};
public:
MonthlyPass(Zone firstZone, Zone secondZone)
{
if (const auto i = zoneIndex(firstZone)) {
coveredZones[*i] = true;
}
if (const auto i = zoneIndex(secondZone)) {
coveredZones[*i] = true;
}
}
bool covers(Zone zone) const
{
const auto i = zoneIndex(zone);
return i.has_value() && coveredZones[*i];
}
};
long long calculateFare(const Journey& journey,
const MonthlyPass* pass = nullptr)
{
const auto index = zoneIndex(journey.zone);
}
```

Implementation:



Task 1 – Objects and OOP

- SmartCard: stores balance; checks and deducts fares. Keeping balance private prevents invalid changes.
- Passenger: stores passenger type; determines concession eligibility.
- Journey: stores zone and travel time.
- FareTable: stores fares and calculates the charge.
- Zone: represents Inner, Middle, or Outer.

OOP groups related data and behaviour into classes; procedural code uses separate variables and functions. Encapsulation protects card balance; abstraction hides fare details. Inheritance and polymorphism could support different concession rules.

Rules: Valid zone and time; peak hours are 8 <= hour < 10 or 17 <= hour < 20; card must retain Rs 50; denied trips do not change the balance. Store money in paise.

Complexity: Procedural and OOP versions both use O(1) time per journey and O(1) memory per card. OOP keeps fare changes and balance rules easier to manage.

Task 2 – Tokens and constants

Examples: keywords class, if; identifiers SmartCard, balance; constants 5000; operators +, >=; punctuators {}, ;.

constexpr int FARES[] = {1000, 2000, 3000};

constexpr int PEAK_PERCENT = 20;

constexpr long long MIN_BALANCE = 5000;

enum class Zone { Inner, Middle, Outer };

constexpr is typed and checked at compile time. enum class prevents accidental integer or unrelated-zone use. Macros lack type and scope checking. Neither choice changes runtime complexity.

Task 3 – Data types

Value	**Type**
Money in paise	long long
Hour, zone index	int
Zone	enum class Zone
Flags	bool
Journey count and totals	long long

Use integer paise for exact money. Three million Rs 36 fares total **10,800,000,000 paise**, which exceeds a typical 32-bit int; use long long.

Record these from your compiler for evidence:

std::cout << sizeof(int) << ' ' << sizeof(long long) << '\n';

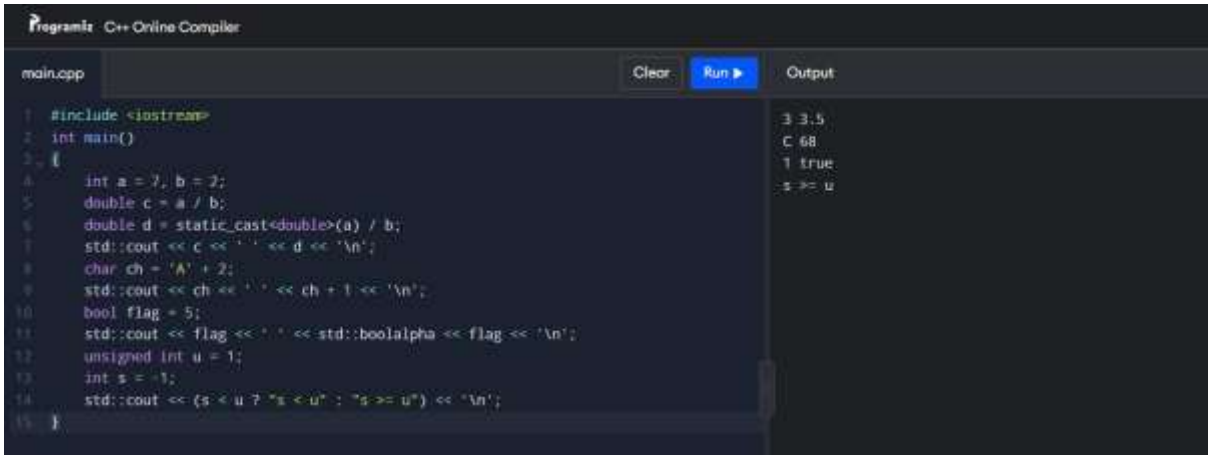
std::cout << INT_MAX << ' ' << LLONG_MAX << '\n';

Both integer and double addition take O(1); integer paise avoids floating-point rounding.

Task 4 – Implicit conversions

Predict first, before running:
3 3.5
C 68
1 true
s >= u

- a / b uses integer division, so 7 / 2 is 3; assigning it to double gives 3.0.
- static_cast<double>(a) / b gives 3.5.
- 'A' + 2 produces 'C'. In ch + 1, char promotes to int, producing 68.
- bool flag = 5 converts nonzero 5 to true; default output is 1, and boolalpha prints true.
- Comparing -1 with unsigned 1 converts -1 to unsigned, making it a large value; therefore s < u is false.



Task 5 – Operators and precedence

Peak check: (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20)
Peak fare: fare += fare * 20 / 100;
Concession: fare -= fare * concessionPercent / 100;

* and / happen before +; parentheses make the intended order clear. x++ uses the old value, while ++x uses the incremented value. Avoid i = i++ because it can produce unexpected results. Integer division truncates toward zero; the remainder has the dividend's sign.



Task 6 – Control structures

Use switch to select the zone's base fare, if to apply peak surcharge, and a loop to process journeys. Skip invalid journeys with continue. A switch default handles unknown zones.

Pseudo code:

FOR each journey

IF zone or hour is invalid

CONTINUE

END IF

```
    SWITCH zone
        Inner: fare = 1000 paise
        Middle: fare = 2000 paise
        Outer: fare = 3000 paise
        DEFAULT: reject journey
    END SWITCH
    IF peak hour
        fare = fare + fare * 20 / 100
    END IF
    IF balance - fare >= 5000
        deduct fare
    ELSE
        deny journey
    END IF
END FOR
```

Task 7 – Procedural vs. object-oriented

Procedural version:

```
bool canTravel(long long balance, long long fare)
{
    return balance - fare >= 5000;
}
long long deductFare(long long balance, long long fare)
{
    return canTravel(balance, fare) ? balance - fare : balance;
}
```

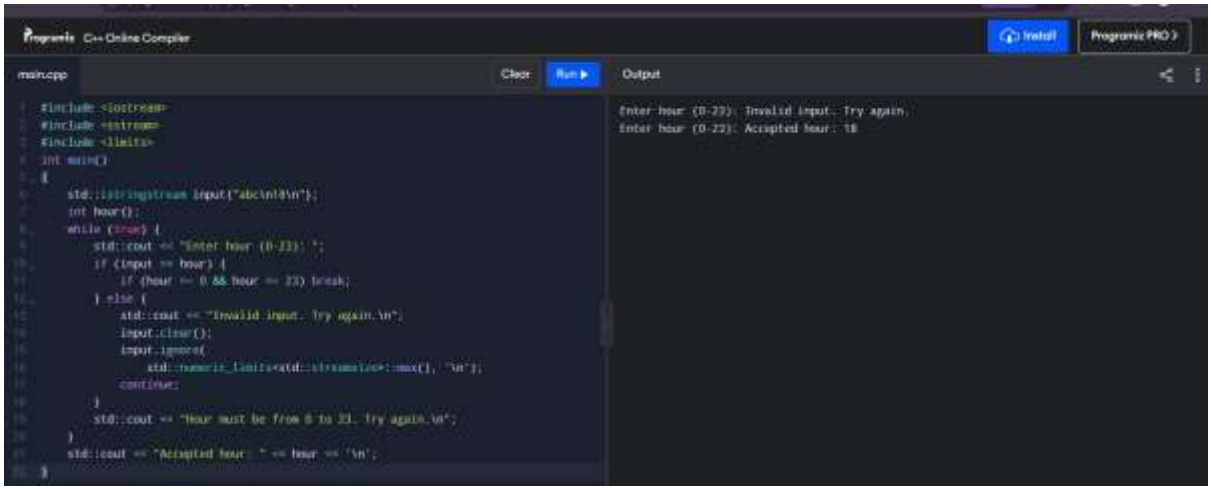
OOP version:

```
class SmartCard {
    long long balance;
public:
```

```
    explicit SmartCard(long long amount) : balance(amount) {}
    bool canTravel(long long fare) const
    {
        return balance - fare >= 5000;
    }
    bool deduct(long long fare)
    {
        if (!canTravel(fare)) return false;
        balance -= fare;
        return true;
    }
    long long getBalance() const { return balance; }
};
```

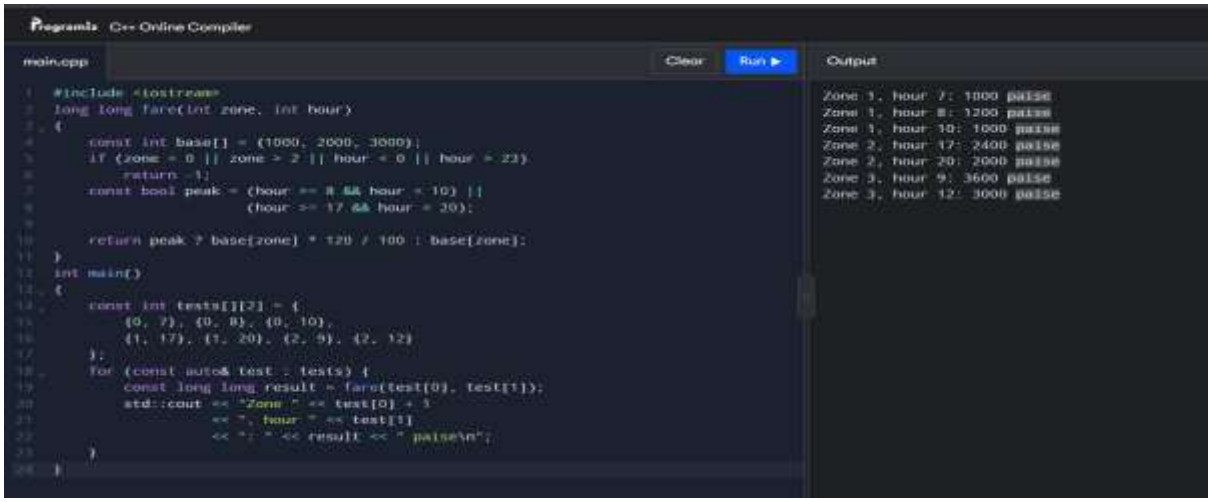
Task 8 – Safe input and output

Check input before using it. If a non-number is entered, clear the error and discard the bad line:



Task 9 – Test the fare logic

This small program tests all three zones at peak and off-peak, plus boundary hours. It uses integer paise.



Task 9 – Fare test cases

The code above only tests zone and hour. The assignment also asks you to test card balance, invalid input, and five journeys' total. Use these cases:

Zone	Hour	Expected fare
Inner	7	Rs 10
Inner	8	Rs 12
Middle	10	Rs 20
Outer	17	Rs 36
Outer	20	Rs 30

Five-journey total: **Rs 108**. Test a card balance of Rs 86 with a Rs 36 fare: it should be denied because only Rs 50 must remain. A balance of Rs 86 with a Rs 36 fare should actually be accepted, leaving exactly Rs 50. Invalid zone, negative recharge, and letters for the hour should be rejected.

Task 10 – Expected vs. actual

For three million Rs 10 fares, the total is **Rs 30,000,000** (3,000,000,000 paise). A typical 32-bit int overflows at that paise total; long long holds it. Integer paise gives an exact total. Record your compiler warnings and measured results after running your program.

Complexity: Each addition is O(1). int, long long, and double use constant space; integer types can overflow, while double can have rounding error.

Task 11 – Final design

Use enum class Zone, constexpr fare constants, integer paise, and a SmartCard class with private balance. Keep fare rules in one function/table. This makes fare updates localized and preserves the Rs 50 minimum.

The design handles millions of journeys because each fare calculation is O(1), but fixed zone logic must change when adding zones or fare rules. Use your test results to support the recommendation.

A configuration file allows fare changes without recompiling, but adds startup file reading O(z), where z is the number of fares, and storage O(z).

Task 12 – Design challenge

Decision: Modify. Add Airport and its Rs 60 fare, apply a 10% weekend discount instead of the peak surcharge, and add a MonthlyPass covering two selected zones.

Use a four-entry fare table and enum class Zone. Update zone validation and tests. The fare calculation checks pass coverage first, then weekend discount, then peak surcharge. SmartCard can remain because its private balance and minimum-balance check still apply; pass-covered trips cost zero.

Example: Airport peak fare = Rs 72. Airport weekend fare = Rs 54, with no peak surcharge. A pass-covered trip costs Rs 0.

The table is easier to extend than a switch, while a separate pass class keeps pass rules out of card-balance logic. Add tests for all zones, peak/weekend combinations, and both pass zones; record actual results from your run.